

0.71.2. La croix en une ligne : `deplace_croix` — doublé, deux positions

Deux lettres, la même croix : deux appels, dans la même image.

1. Ce qu'il faut comprendre

C'est le **0.71** et le **0.71.1 réunis** : deux appels, un par lettre.

Les deux bougent dans la même image : la console ne fait passer le temps d'attente qu'une fois par image ; chaque appel de cette image a le droit de faire son pas.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1781 octets** de cartouche.

1. UNE VARIABLE — X

```
uint8_t x = 9;      // le A
```

2. UNE VARIABLE — Y

```
uint8_t y = 8;
```

3. UNE VARIABLE — XB

```
uint8_t xb = 4;     // le B
```

4. UNE VARIABLE — YB

```
uint8_t yb = 4;
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

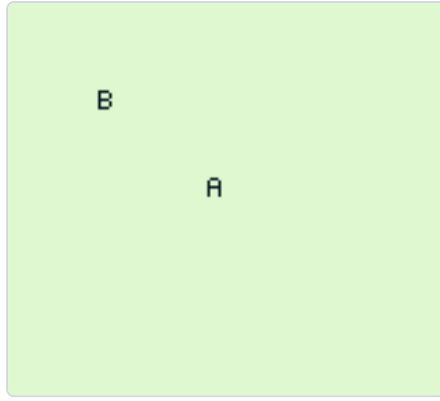
```
int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250);
        deplace_croix(xb, yb, ALPHABET[1], 250);
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Le A et le B suivent la croix ensemble.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le A a fait 4 pas
- ✓ le B aussi, dans la même image

Ensuite : **0.72. Plus vite sur la grille : deplace_croix à 100** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++